Automating Network Configuration with Micetro

Overview

Micetro is an extremely powerful orchestration tool. Providing a single GUI/API across disparate DNS & DHCP systems, together with discovery and IPAM. Giving that single pane of glass view into the entire network.
This works great for centralising administration, audit logging plus wrapping around Role Based Access even for DNS and DHCP platforms that don't natively support this.

What isn't necessarily fully appreciated (though I know some customers are doing this!) is how you can leverage the Micetro API, together with the DDI data Micetro has visibility into to streamline business processes.

In order to highlight this, I've put together some sample Python code, built around the Micetro API to automate the provisioning of network interfaces on some VyOS devices.
[VyOS (https://vyos.io) is an open source router/firewall. The command syntax is very similar to Junos OS]

Although this example is written for VyOS I've tried to make the code easy to adapt for other platforms.

→ https://github.com/worrabkram/Micetro_Automation_example

APIs

Micetro has several API options including integrations for Ansible and Terraform. More details here → https://docs.bluecatnetworks.com/r/en-US/Micetro-User-Guide/Automation/25.2.0

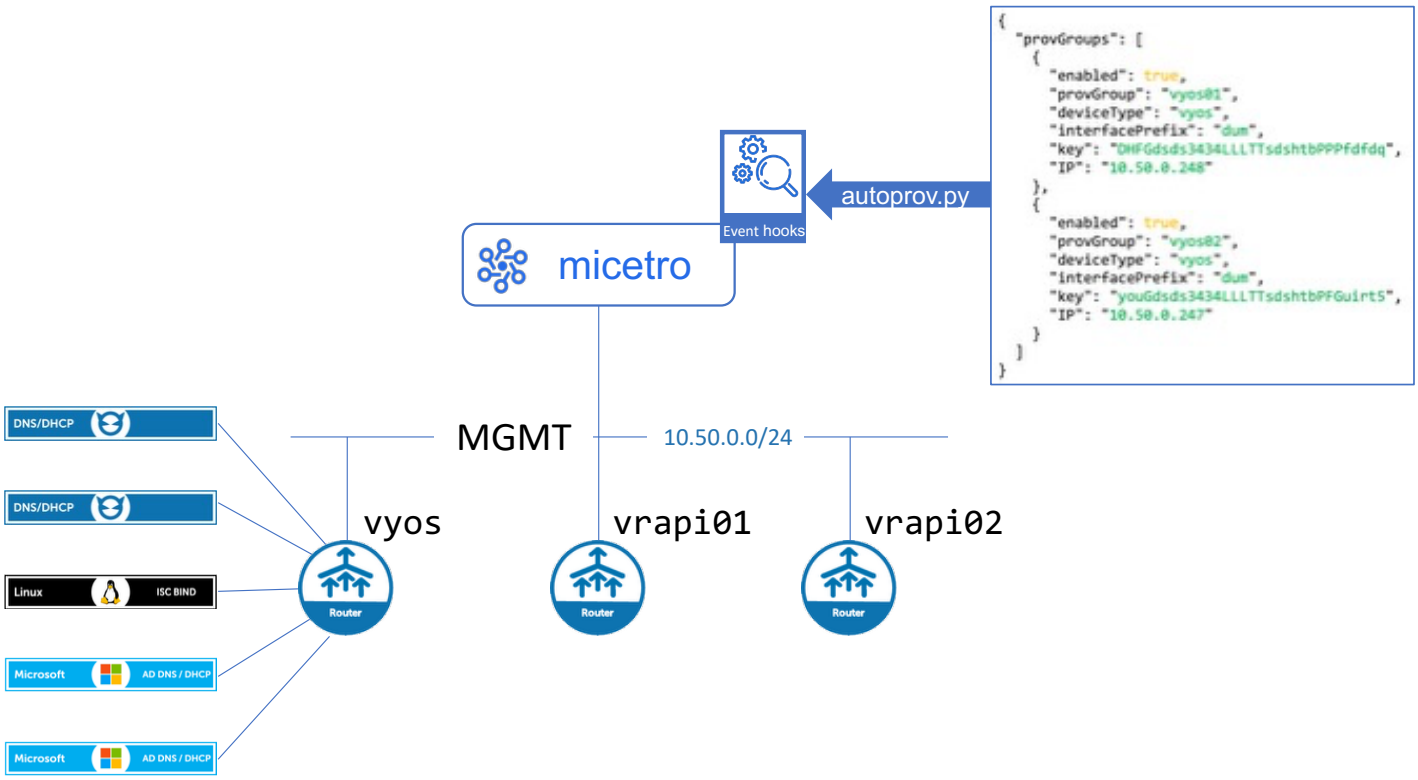
For this example I'm using the JSON-RPC API which I'm most familiar with. However, for new customers I usually point them at the swagger interface bundled with Micetro as this is a great way of trying out/testing API queries → http(s)://<micetro.yourdomain.tld>/mmws/api/doc

For configuring VyOS I'm using the VyOS API together with the pyvyos library for Python
https://pypi.org/project/pyvyos/

My Environment

My Lab environment for this example consists of the following infrastructure

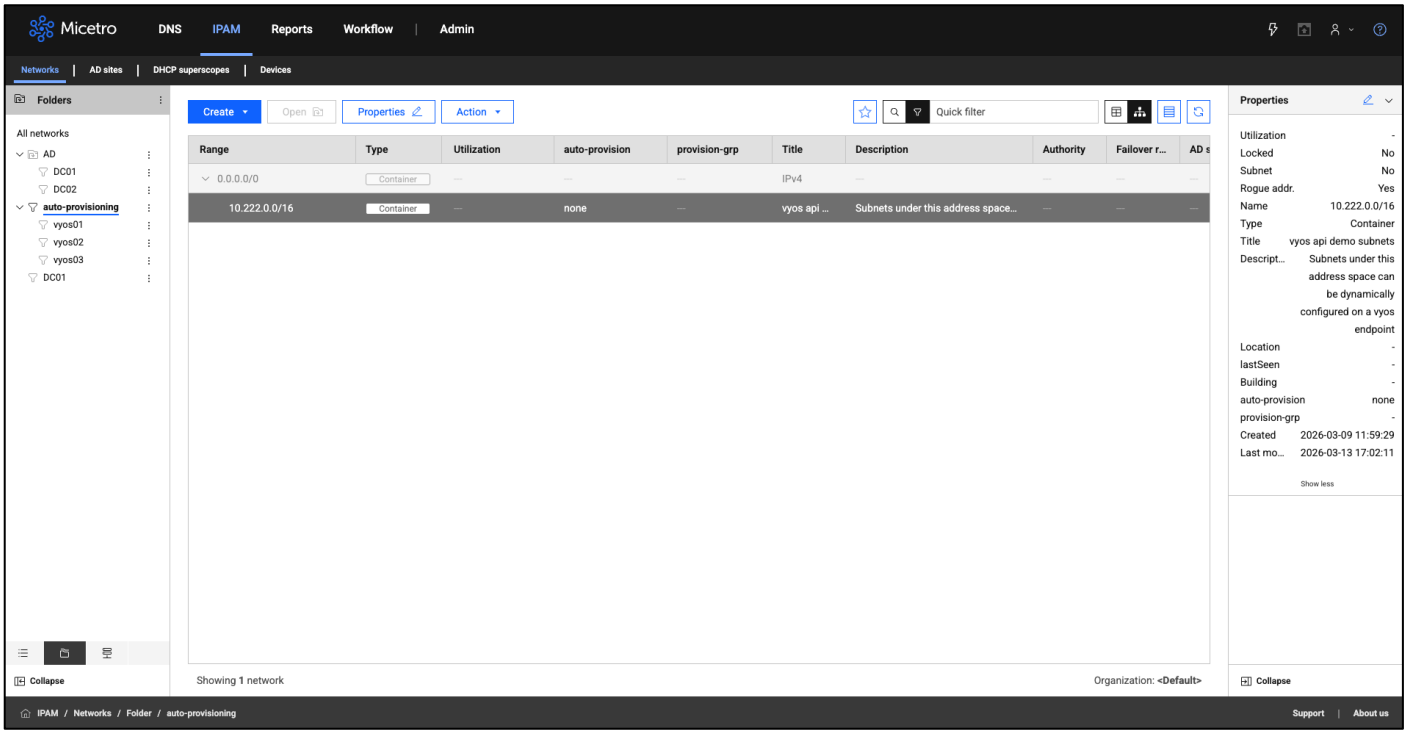
- All Micetro components installed on a single Windows 2019 server
- One virtirtual VyOS, vyos providing routing between all components
- A linux BIND server managed by Micetro
- 2x Windows AD DCs running MS DNS/DHCP managed by Micetro
- 2x additional VyOS routers connected to a shared Management Network
- 2x BlueCat Appliances managed by Micetro



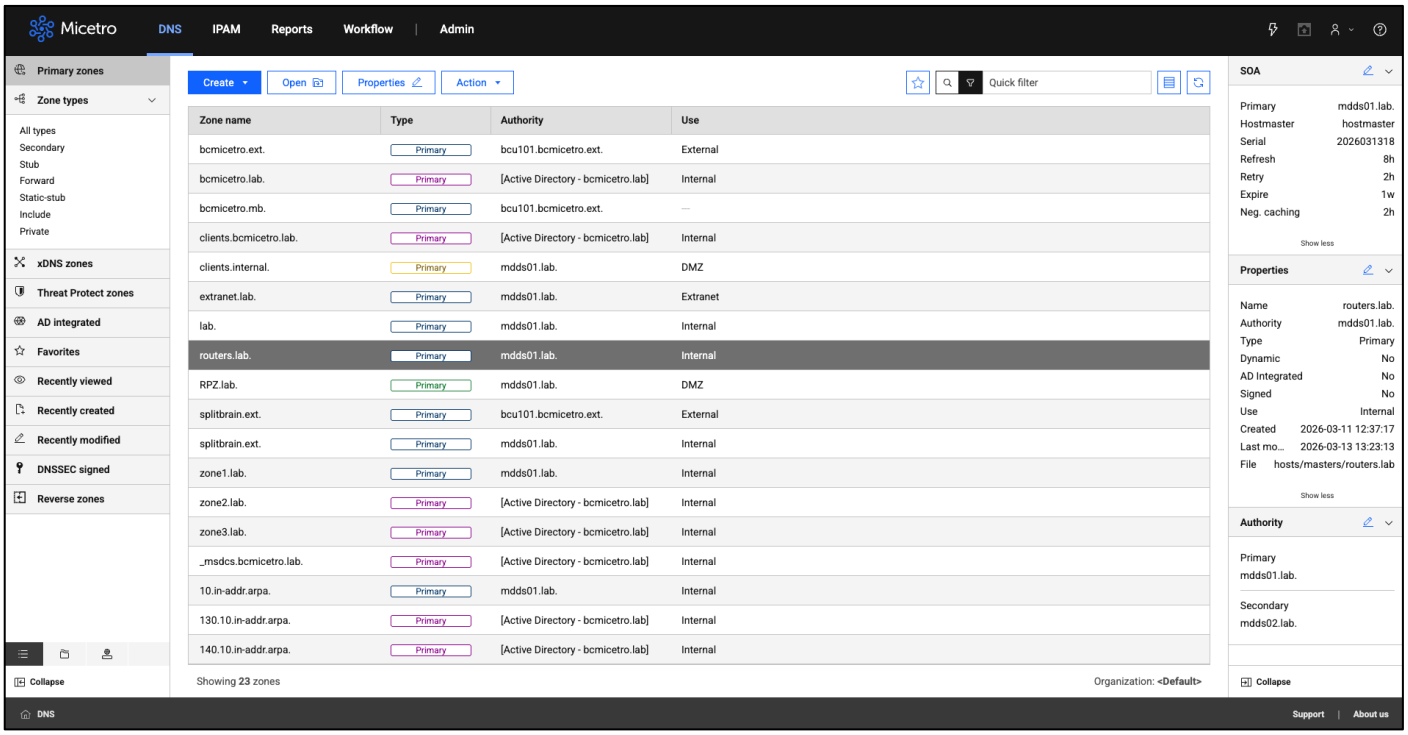

```
{
  "provGroups": [
    {
      "enabled": true,
      "provGroup": "vyos01",
      "deviceType": "vyos",
      "interfacePrefix": "dum",
      "key": "DHFGdsds3434LLLTTsdshtbPPPfdfdq",
      "IP": "10.50.0.248"
    },
    {
      "enabled": true,
      "provGroup": "vyos02",
      "deviceType": "vyos",
      "interfacePrefix": "dum",
      "key": "youGdsds3434LLLTTsdshtbPFGuirt5",
      "IP": "10.50.0.247"
    }
  ]
}
```


Micetro config

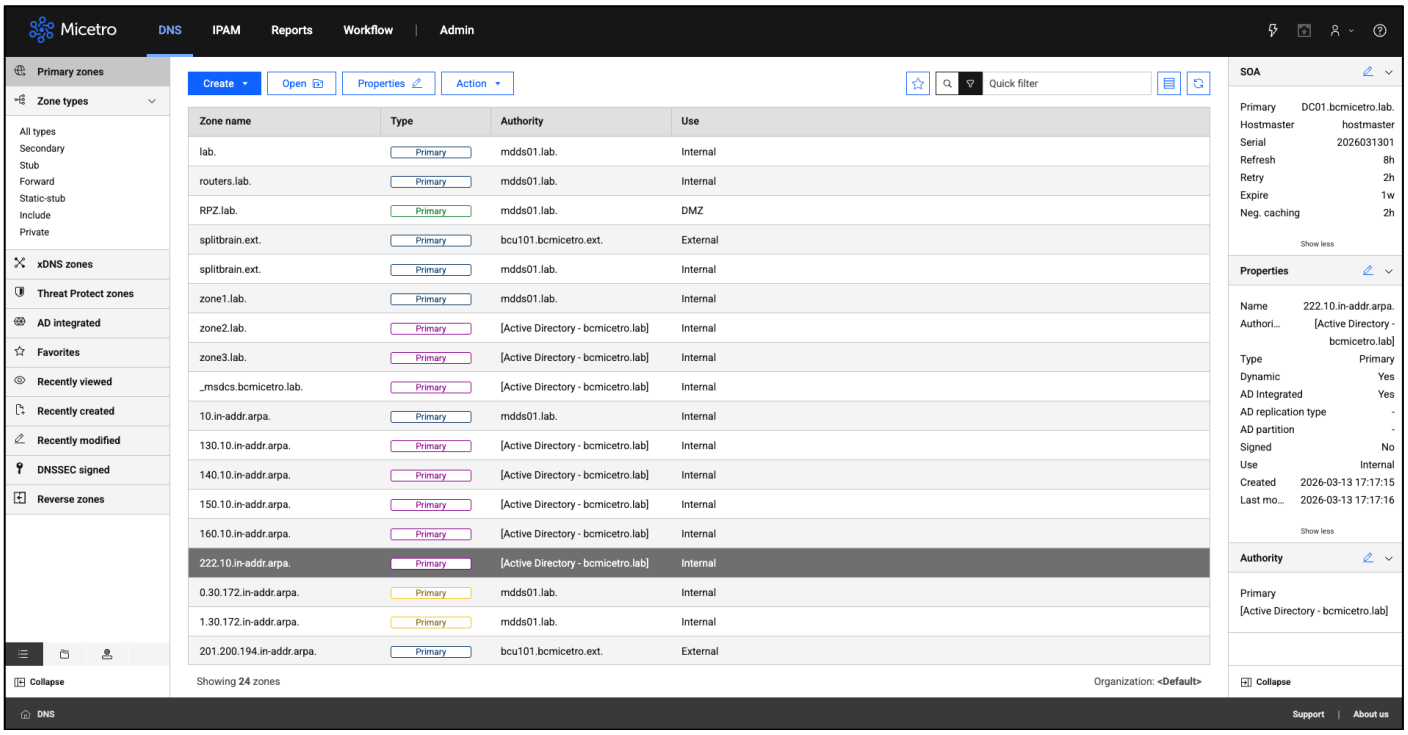
- I have an IPAM Container '10.222.0.0/16' which I'll use to allocate networks to my VyOS routers



- A DNS zone ‘routers.lab’ allocated to my BlueCat appliances which I’ll use to store the forward entries for any related gateway interfaces created



- And a DNS reverse zone ‘222.10.in-addr.arpa.’ allocated to AD DNS which I’ll use to store the reverse entries for any related gateway interfaces created



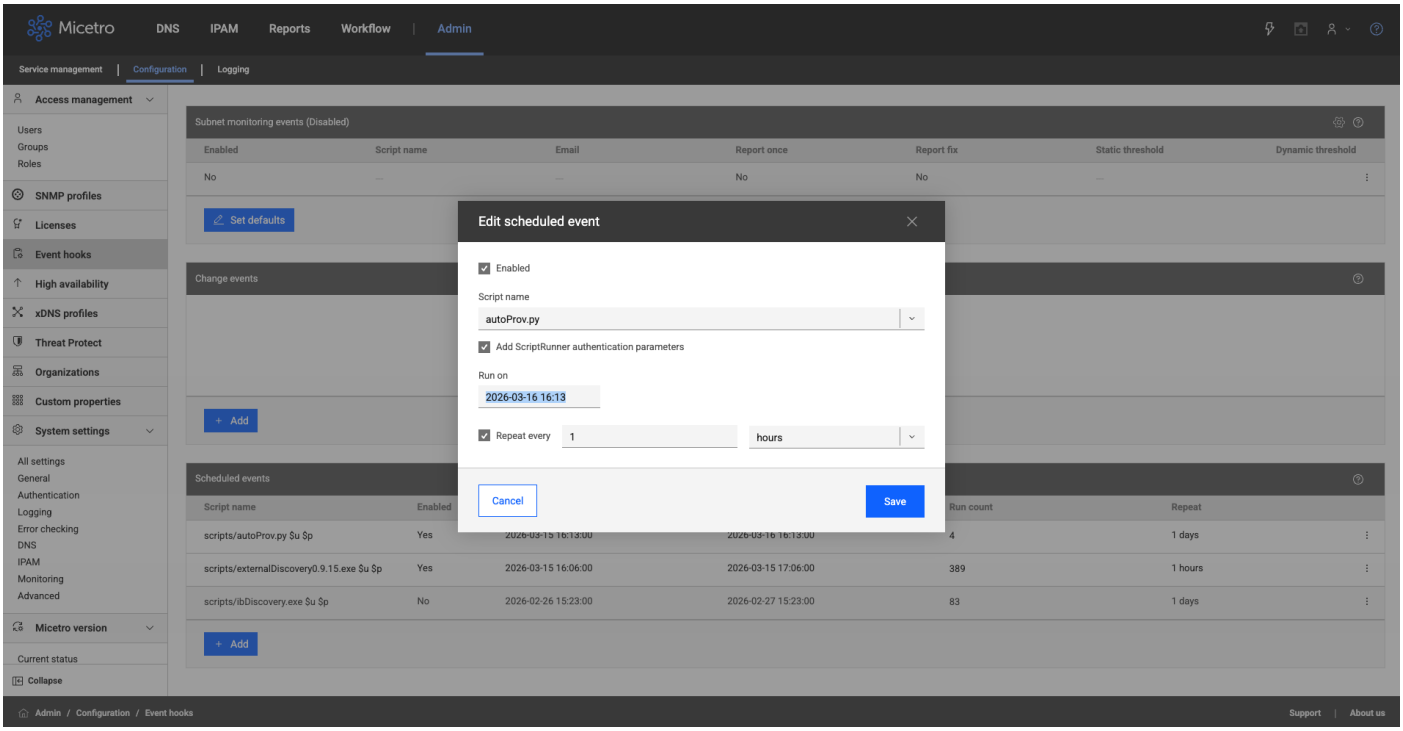
Automation Script Components

Downloadable from GitHub
https://github.com/worrabkram/Micetro_Automation_example

autoProv.py	Example automation script written in Python
autoProv.exe	x64 Windows binary/package created using pyinstaller
autoProvSetup.json	JSON format configuration file. Needs to be tailored for the target environment
autoProvGroups.json	JSON format file containing target router info Needs to be tailored for the target environment
INSTALL.pdf	Install and configuration instructions
Automating_Network_Configuration_with_Micetro.pdf	This document

The automation script needs to be deployed to Micetro as an Event Hook. This is detailed in the INSTALL.pdf accompanying the script.

More info on Micetro Event hooks can be found here
→ https://docs.bluecatnetworks.com/r/en-US/Micetro-Admin-Guide/Event-hooks/25.2.0



AutoProv.py set to run every hour

A Working Example

Router interfaces [pre-provisioning]

Configuration Group vyos01

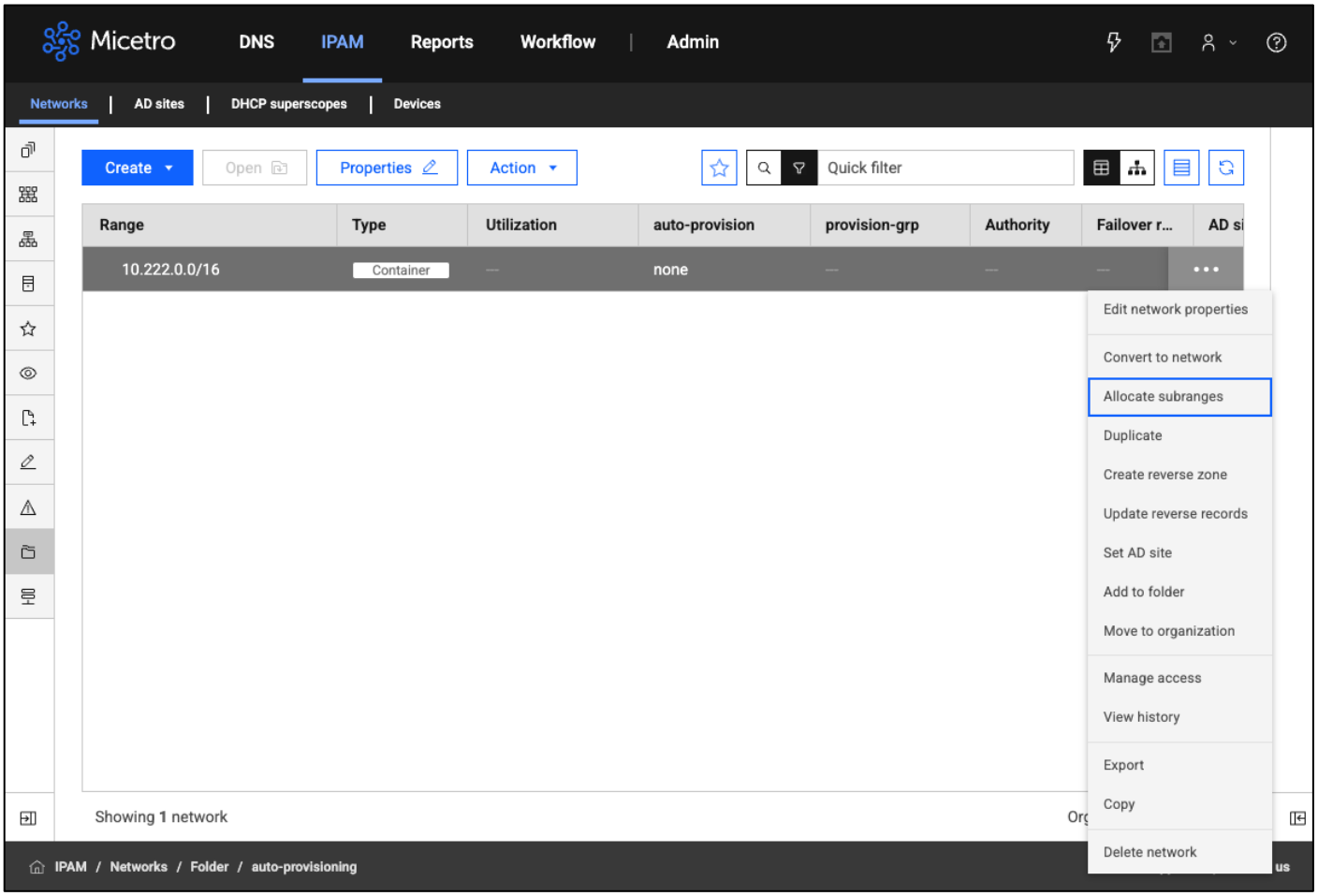
```
vyos@vrapi01:~$ show interfaces
Codes: S - State, L - Link, u - Up, D - Down, A - Admin Down
Interface    IP Address      MAC                VRF        MTU  S/L    Description
-----------  --------------  -----------------  -------  -----  -----  -------------
eth0         10.50.0.248/24  bc:24:11:7a:98:ad  default   1500  u/u    vyos api MGMT
eth1         -               bc:24:11:f3:41:da  default   1500  u/u
lo           127.0.0.1/8     00:00:00:00:00:00  default  65536  u/u
             ::1/128
```

Configuration Group vyos02

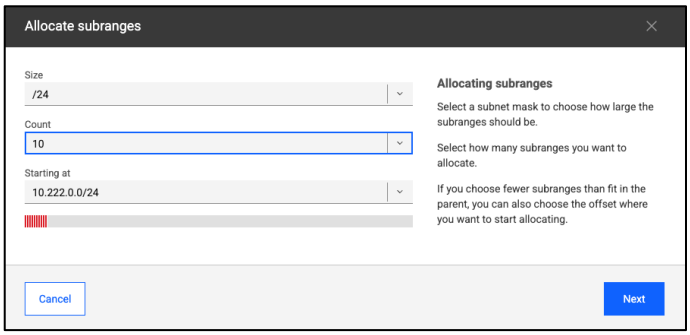
```
vyos@vrapi02:~$ show interfaces
Codes: S - State, L - Link, u - Up, D - Down, A - Admin Down
Interface    IP Address      MAC                VRF        MTU  S/L    Description
-----------  --------------  -----------------  -------  -----  -----  -------------
eth0         10.50.0.247/24  bc:24:11:57:19:dc  default   1500  u/u    MGMT
eth1         -               bc:24:11:2a:09:29  default   1500  u/u
lo           127.0.0.1/8     00:00:00:00:00:00  default  65536  u/u
             ::1/128
```

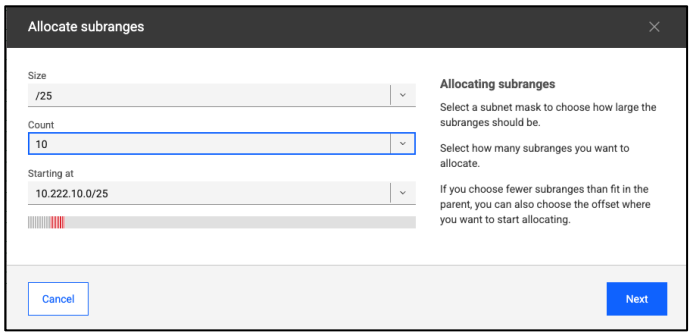
Creating Some Networks to Provision to my Routers

- Using the Allocate subranges utility to create some new ranges under the /16 parent container

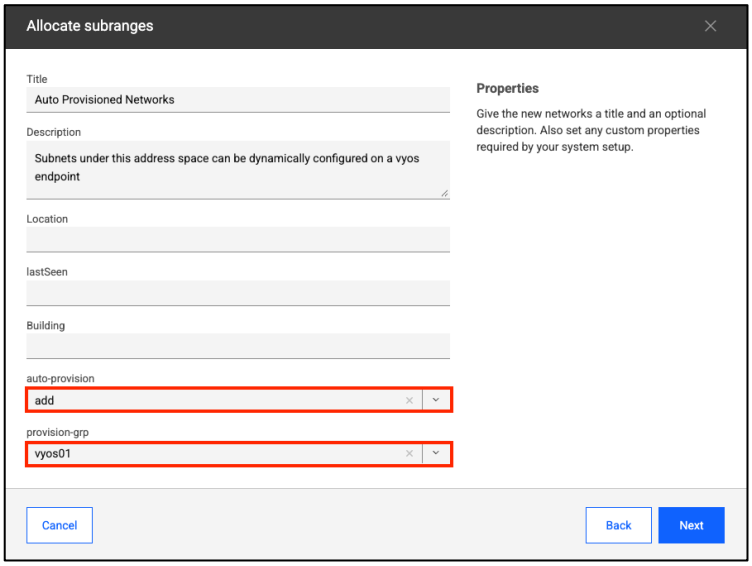


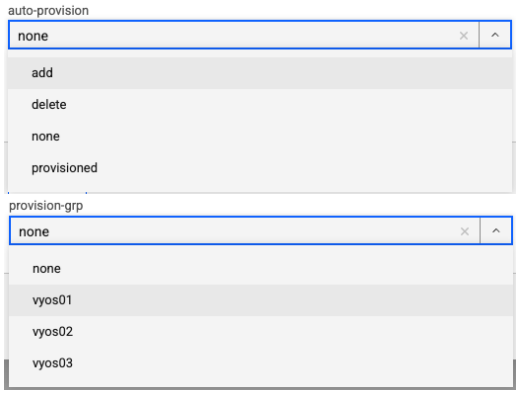
- Creating 10x /24 ranges to allocate to the vyos01 Group and 10x /25 ranges for the vyos02 Group



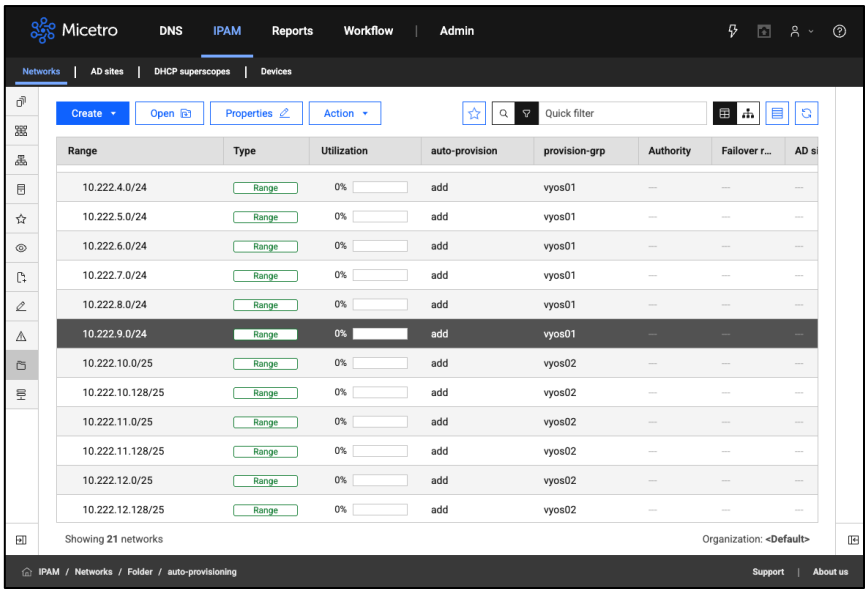


- Selecting ‘add’ for the custom property ‘auto-provision’
- Selecting ‘vyos01’ → /24 ranges and ‘vyos02’ → /25 ranges for the ‘provision-grp’ custom property

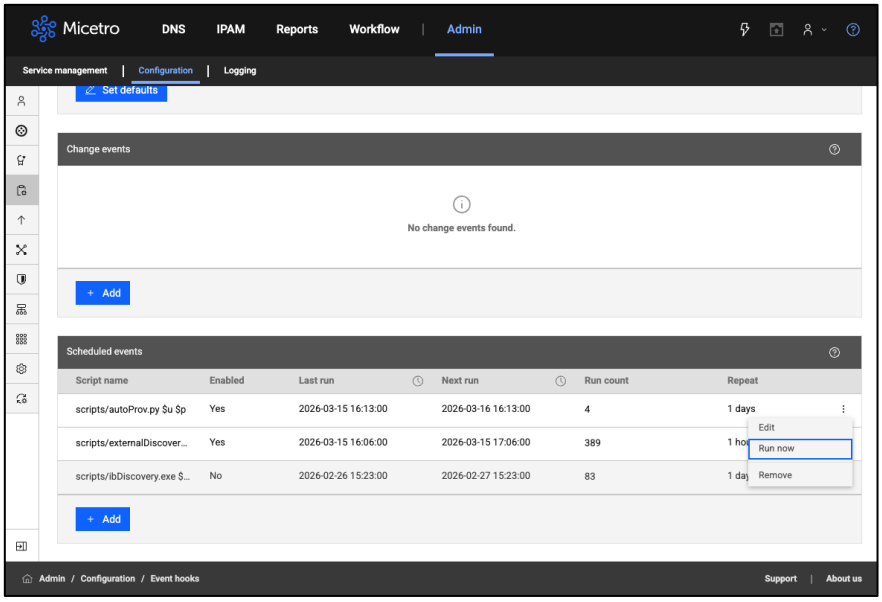




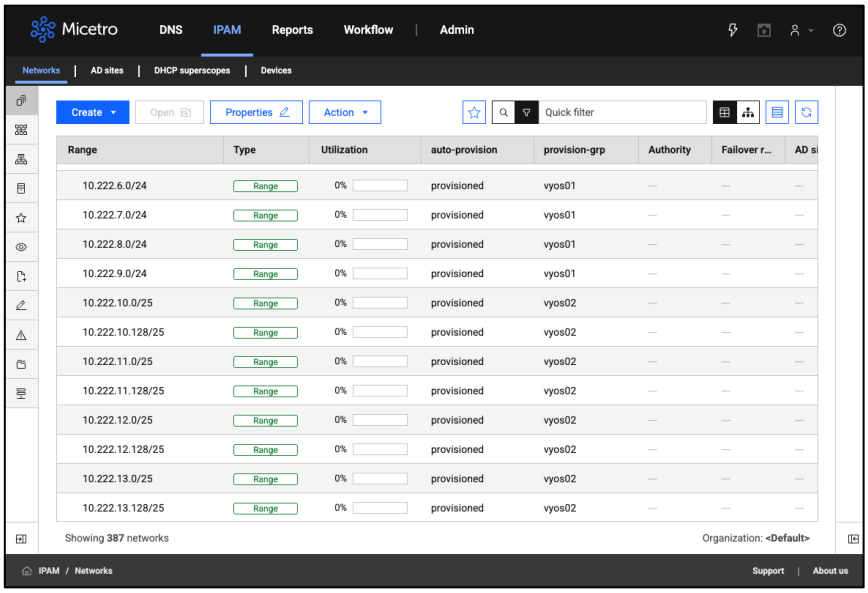
- 20 new ranges are now displayed. 10 allocated to vyos01 Group and 10 to the vyos02 Group. All set to ‘add’



- We could just wait for the next hourly scheduled run of the provisioning script (or perhaps you'd just want to run this overnight?). However, there is also an option to 'Run now'



- After the provisioning script has run. Provided the interfaces were successfully added the routers the 'auto-provision' Custom property should get updated to 'provisioned'



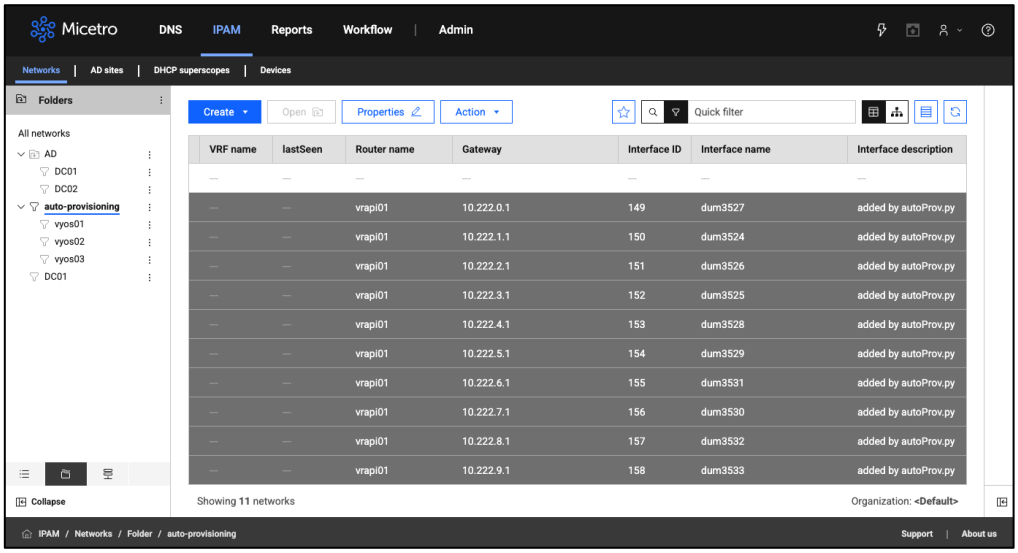
- Checking the routers

```
vyos@vrapi01:~$ show interfaces
Codes: S - State, L - Link, u - Up, D - Down, A - Admin Down
Interface    IP Address       MAC               VRF        MTU  S/L    Description
-----------  --------------   ----------------  -------  -----  -----  --------------------
dum3524      10.222.1.1/24    e6:52:53:0c:e7:1a  default   1500  u/u    added by autoProv.py
dum3525      10.222.3.1/24    c2:eb:43:7c:51:4b  default   1500  u/u    added by autoProv.py
dum3526      10.222.2.1/24    d2:3b:a6:b9:34:8c  default   1500  u/u    added by autoProv.py
dum3527      10.222.0.1/24    ba:3f:41:4b:c5:be  default   1500  u/u    added by autoProv.py
dum3528      10.222.4.1/24    36:fe:10:78:8e:d9  default   1500  u/u    added by autoProv.py
dum3529      10.222.5.1/24    ae:04:26:cf:3c:fb  default   1500  u/u    added by autoProv.py
dum3530      10.222.7.1/24    72:dd:d3:d5:ba:1b  default   1500  u/u    added by autoProv.py
dum3531      10.222.6.1/24    0e:db:bd:d1:c7:9e  default   1500  u/u    added by autoProv.py
dum3532      10.222.8.1/24    3e:25:c9:58:20:56  default   1500  u/u    added by autoProv.py
dum3533      10.222.9.1/24    7a:7d:52:63:09:c8  default   1500  u/u    added by autoProv.py
eth0         10.50.0.248/24   bc:24:11:7a:98:ad  default   1500  u/u    vyos api MGMT
eth1         -                bc:24:11:f3:41:da  default   1500  u/u
lo           127.0.0.1/8      00:00:00:00:00:00  default  65536  u/u
             ::1/128
```

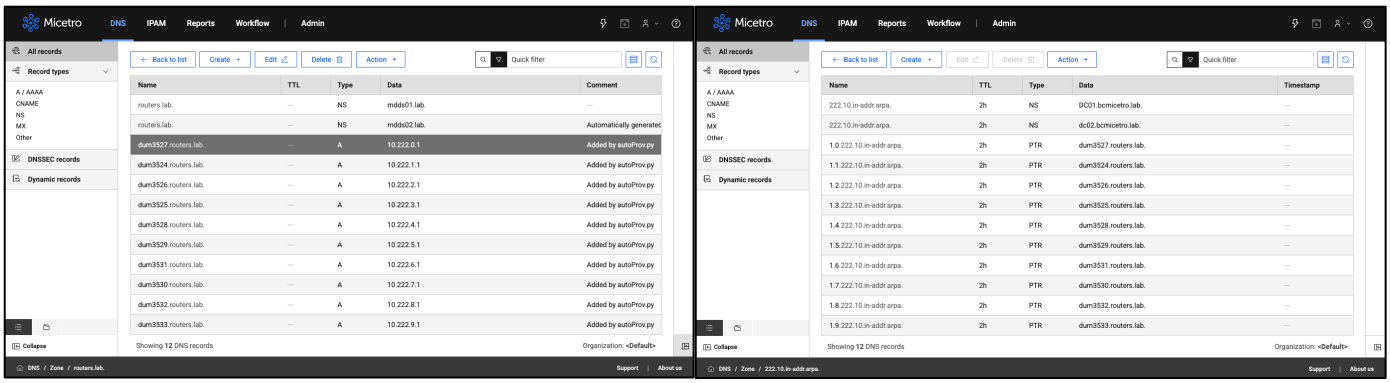
```
vyos@vrapi02:~$ show interfaces
Codes: S - State, L - Link, u - Up, D - Down, A - Admin Down
Interface    IP Address        MAC               VRF        MTU  S/L    Description
-----------  ----------------  ----------------  -------  -----  -----  -------------------
dum3514      10.222.12.1/25    76:94:58:c1:b3:6d  default   1500  u/u    added by autoProv.py
dum3515      10.222.11.129/25  7a:15:fb:5e:0d:b1  default   1500  u/u    added by autoProv.py
dum3516      10.222.11.1/25    b6:63:0c:08:9d:73  default   1500  u/u    added by autoProv.py
dum3517      10.222.10.1/25    7a:54:fd:ee:31:de  default   1500  u/u    added by autoProv.py
dum3518      10.222.10.129/25  2e:24:09:8d:3e:f8  default   1500  u/u    added by autoProv.py
dum3519      10.222.12.129/25  de:bd:27:09:7c:c2  default   1500  u/u    added by autoProv.py
dum3520      10.222.13.1/25    66:85:52:b8:04:a5  default   1500  u/u    added by autoProv.py
dum3521      10.222.14.1/25    7e:48:b4:46:62:73  default   1500  u/u    added by autoProv.py
dum3522      10.222.13.129/25  9a:fe:74:0c:a4:e3  default   1500  u/u    added by autoProv.py
dum3523      10.222.14.129/25  12:ec:37:47:ad:9d  default   1500  u/u    added by autoProv.py
eth0         10.50.0.247/24    bc:24:11:57:19:dc  default   1500  u/u    MGMT
eth1         -                 bc:24:11:2a:09:29  default   1500  u/u
lo           127.0.0.1/8       00:00:00:00:00:00  default  65536  u/u
             ::1/128
```

As can be seen from the above the new interfaces have been added to both vrapi01(vyos01 Group) and vrapi02(vyos02 Group)

- If you have Micetro discovery enabled the new interface details also get read into the GUI



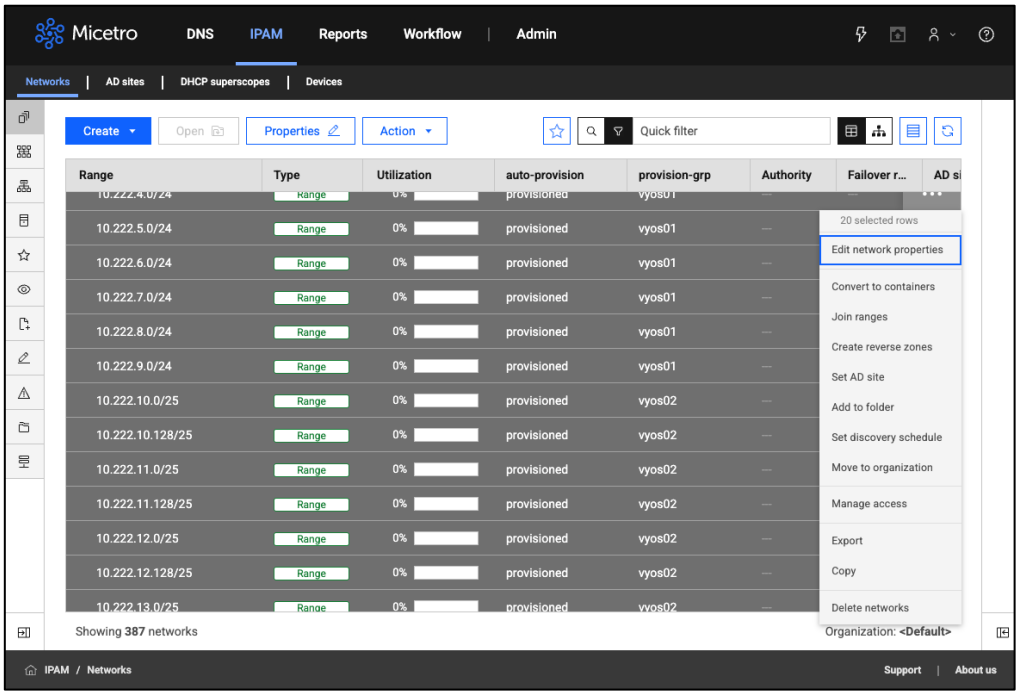
- Forward and reverse DNS entries can also be created automatically



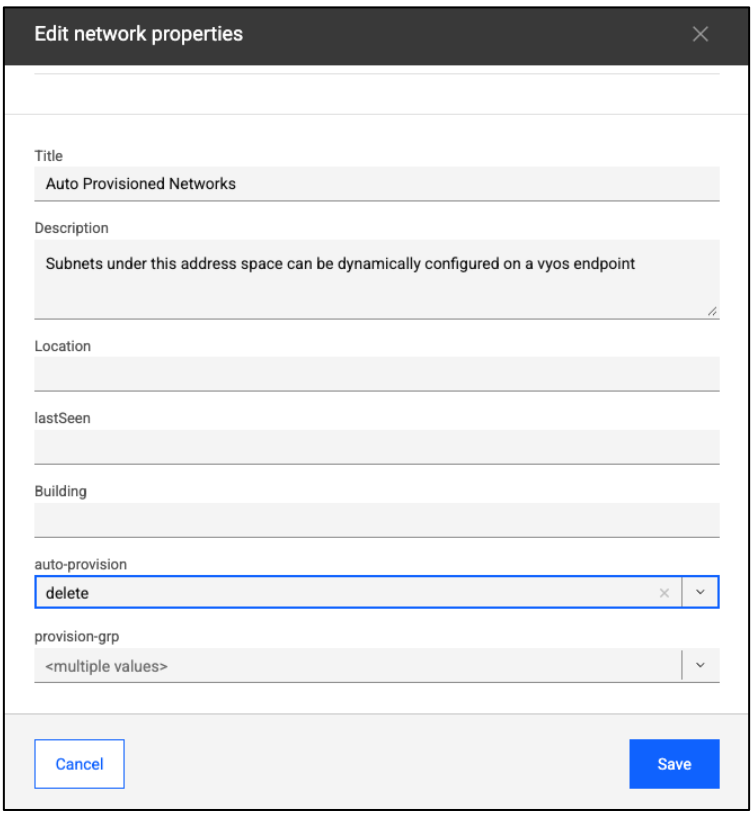
The Change in reverse

The provisioning script can also be used to de-commission network interfaces as follows-

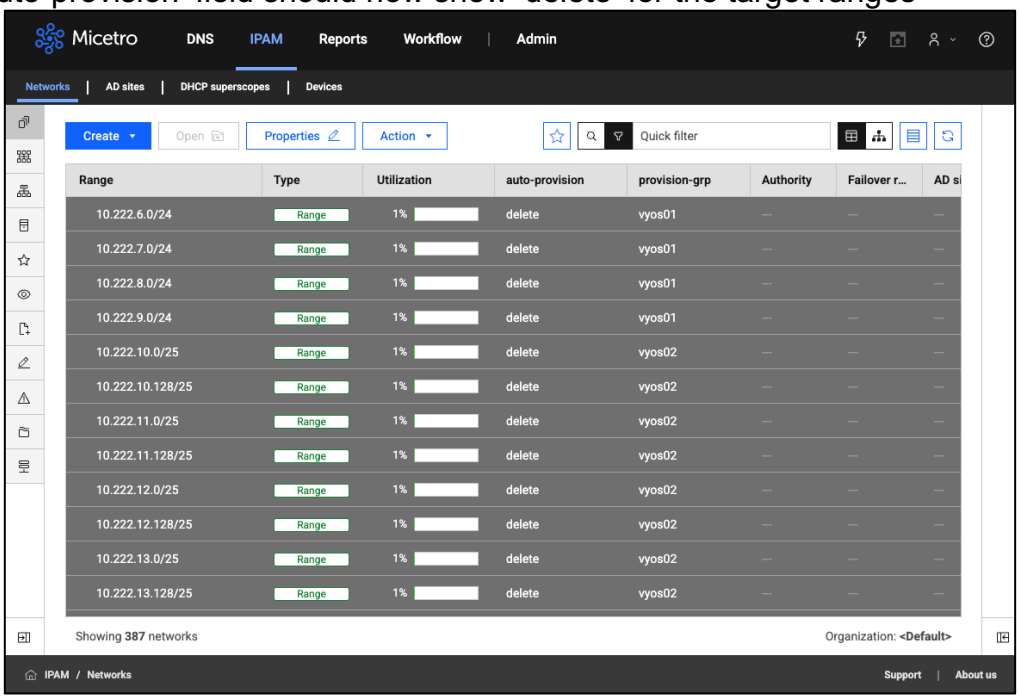
- Select the networks in Micetro you want to de-commission. Then from the dropdown 'Edit Network properties'



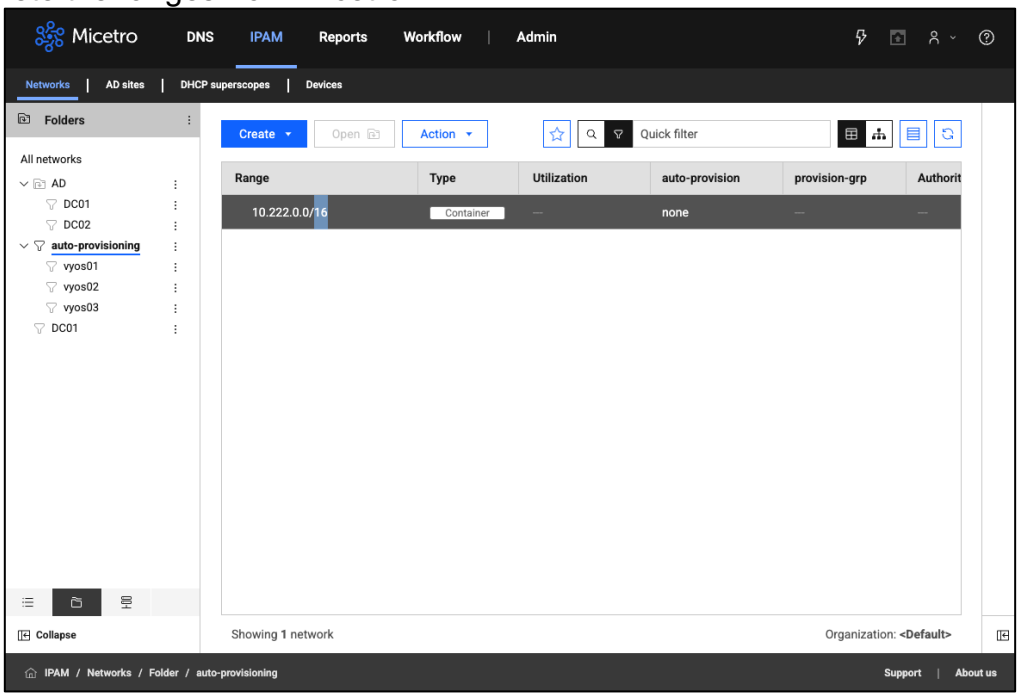
- Edit the 'auto-provision' custom property and set to 'delete'



- The ‘auto-provision’ field should now show ‘delete’ for the target ranges



- Either wait for the next scheduled run, or click ‘Run now’ against the ‘Scheduled Event hook’
- The provisioning script should now remove the network interfaces from the target routers and delete the ranges from Micetro

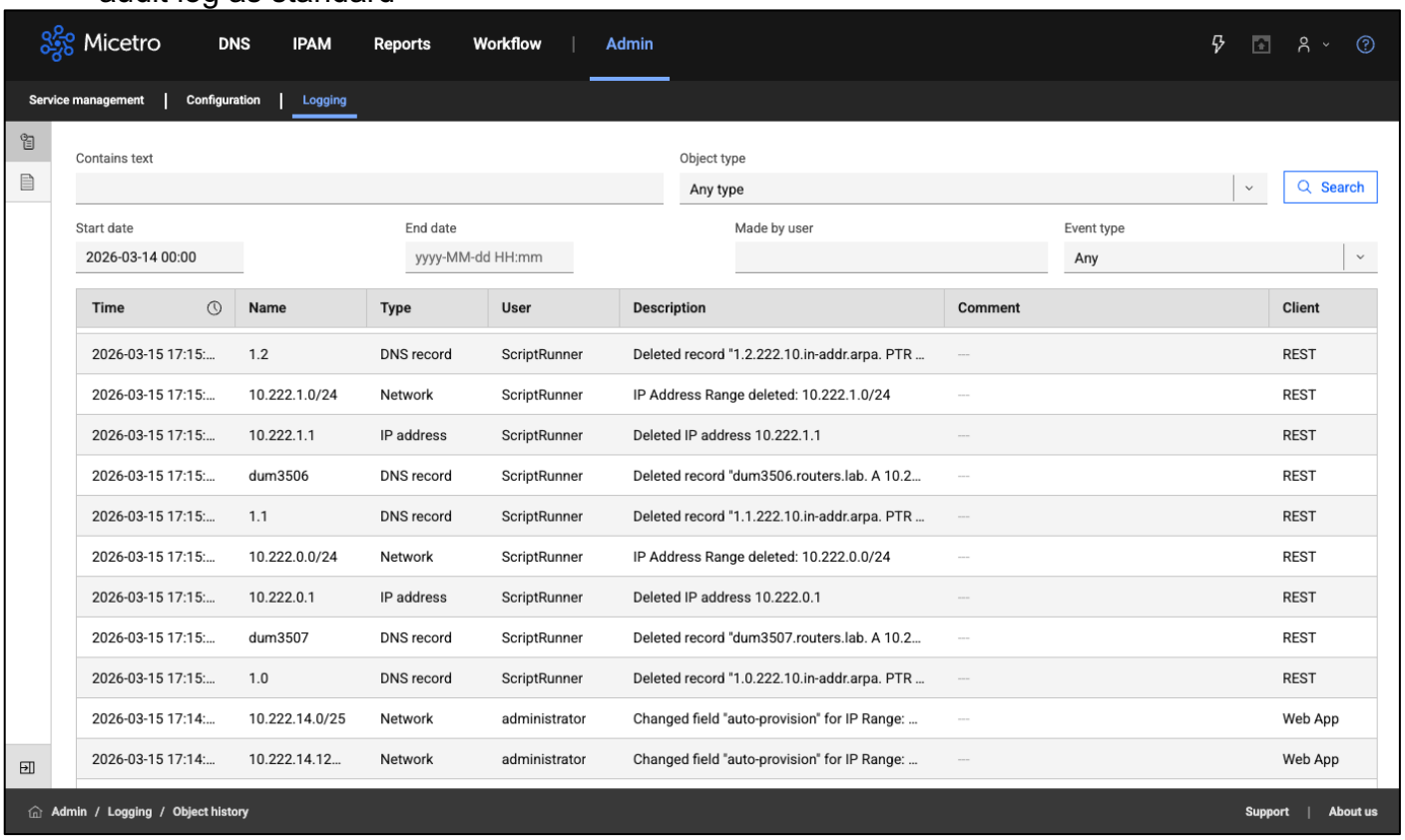


- Confirming the Interfaces have been removed from the routers

```
vyos@vrapi01:~$ show interfaces
Codes: S - State, L - Link, u - Up, D - Down, A - Admin Down
Interface     IP Address       MAC                VRF        MTU  S/L    Description
-----------   --------------   -----------------  -------   -----  -----  -------------
eth0          10.50.0.248/24   bc:24:11:7a:98:ad  default    1500  u/u    vyos api MGMT
eth1          -                bc:24:11:f3:41:da  default    1500  u/u
lo            127.0.0.1/8      00:00:00:00:00:00  default   65536  u/u
              ::1/128
```

```
vyos@vrapi02:~$ show interfaces
Codes: S - State, L - Link, u - Up, D - Down, A - Admin Down
Interface    IP Address      MAC                VRF        MTU  S/L    Description
-----------  --------------  -----------------  -------  -----  -----  -------------
eth0         10.50.0.247/24  bc:24:11:57:19:dc  default   1500  u/u    MGMT
eth1         -               bc:24:11:2a:09:29  default   1500  u/u
lo           127.0.0.1/8     00:00:00:00:00:00  default  65536  u/u
             ::1/128
```

- In line with all other DDI related changes. All of these activities get logged to the Micetro audit log as standard



Time	Name	Type	User	Description	Comment	Client
2026-03-15 17:15:...	1.2	DNS record	ScriptRunner	Deleted record "1.2.222.10.in-addr.arpa. PTR ...	---	REST
2026-03-15 17:15:...	10.222.1.0/24	Network	ScriptRunner	IP Address Range deleted: 10.222.1.0/24	---	REST
2026-03-15 17:15:...	10.222.1.1	IP address	ScriptRunner	Deleted IP address 10.222.1.1	---	REST
2026-03-15 17:15:...	dum3506	DNS record	ScriptRunner	Deleted record "dum3506.routers.lab. A 10.2...	---	REST
2026-03-15 17:15:...	1.1	DNS record	ScriptRunner	Deleted record "1.1.222.10.in-addr.arpa. PTR ...	---	REST
2026-03-15 17:15:...	10.222.0.0/24	Network	ScriptRunner	IP Address Range deleted: 10.222.0.0/24	---	REST
2026-03-15 17:15:...	10.222.0.1	IP address	ScriptRunner	Deleted IP address 10.222.0.1	---	REST
2026-03-15 17:15:...	dum3507	DNS record	ScriptRunner	Deleted record "dum3507.routers.lab. A 10.2...	---	REST
2026-03-15 17:15:...	1.0	DNS record	ScriptRunner	Deleted record "1.0.222.10.in-addr.arpa. PTR ...	---	REST
2026-03-15 17:14:...	10.222.14.0/25	Network	administrator	Changed field "auto-provision" for IP Range: ...	---	Web App
2026-03-15 17:14:...	10.222.14.12...	Network	administrator	Changed field "auto-provision" for IP Range: ...	---	Web App